Shortest Common Supersequence

The shortest common supersequence (SCS) is the problem of finding the shortest supersequence Z of given sequences X and Y such that both X & Y are subsequences of Z.

Example:

```
Input:   str1 = "geek",  str2 = "eke"
Output: "geeke"

Input:   str1 = "AGGTAB",  str2 = "GXTXAYB"
Output:  "AGXGTXAYB"
```

Solution 1

This problem is closely related to longest common subsequence problem. Below are steps.

1) Find Longest Common Subsequence (LCS) of two given strings. For example, LCS of “geek” and “eke” is “ek”.

2) Insert non- LCS characters (in their original order in strings) to the LCS found above, and return the result. So “ek” becomes “geeke” which is shortest common supersequence.

Let us consider another example, str1 = “AGGTAB” and str2 = “GXTXAYB”. LCS of str1 and str2 is “GTAB”. Once we find LCS, we insert characters of both strings in order and we get “AGXGTXAYB”

How does this work?

We need to find a string that has both strings as subsequences and is shortest such string. If both strings have all characters different, then result is sum of lengths of two given strings. If there are common characters, then we don’t want them multiple times as the task is to minimize length. Therefore, we fist find the longest common subsequence, take one occurrence of this subsequence and add extra characters.

```
Length of the shortest supersequence = (Sum of lengths of given two strings) -
                                        (Length of LCS of two given strings)
```

Solution 2

The SCS problem has an optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can be broken down into yet simpler subproblems, and so on, until, finally, the solution becomes trivial.

1. Let us consider two sequences X and Y of length m and n respectively that both end in the same element.

To find their SCS, we shorten each sequence by removing the last element, find the SCS of the shortened sequences, and to that SCS append the removed element. So we can say that

```
SCS(X[1..m], Y[1..n]) = SCS(X[1..m-1], Y[1..n-1]) + X[m]
                                                    if X[m] = Y[n]
```

For example,

```
SCS('ABCBDA', 'BDCABA') = SCS('ABCBD', 'BDCAB') + 'A'
SCS('ABCBDAB', 'BDCAB') = SCS('ABCBDA', 'BDCA') + 'B'
```

2. Now suppose that the two sequences do not end in the same element.

Then the SCS of X and Y is the **shorter** of the two sequences

`SCS(X[1..m-1], Y[1..n]) + X[m] and`

`SCS(X[1..m], Y[1..n-1]) + Y[n].`

To understand this property, let's consider the two following sequences

```
X: ABCBDAB (n elements)
Y: BDCABA (m elements)
```

The SCS of these two sequences either ends with a B (the last element of sequence X) or with A (the last element of sequence Y).

Case 1: If SCS ends with a B, then we add B from sequence X to SCS and the problem reduces to `SCS(X[1..m-1], Y[1..n]) + X[m].`

Case 2: If SCS does not end with a B, that means it ends with A, then we add A from sequence Y to SCS and the problem reduces to `SCS(X[1..m], Y[1..n-1]) + Y[n].`

For example,

```
SCS('ABCBD', 'BDCAB') = minimum (SCS('ABCB', 'BDCAB') + 'D',
                                 SCS('ABCBD', 'BDCA') + 'B')
```

Below solution finds the length of shortest common supersequence of sequences

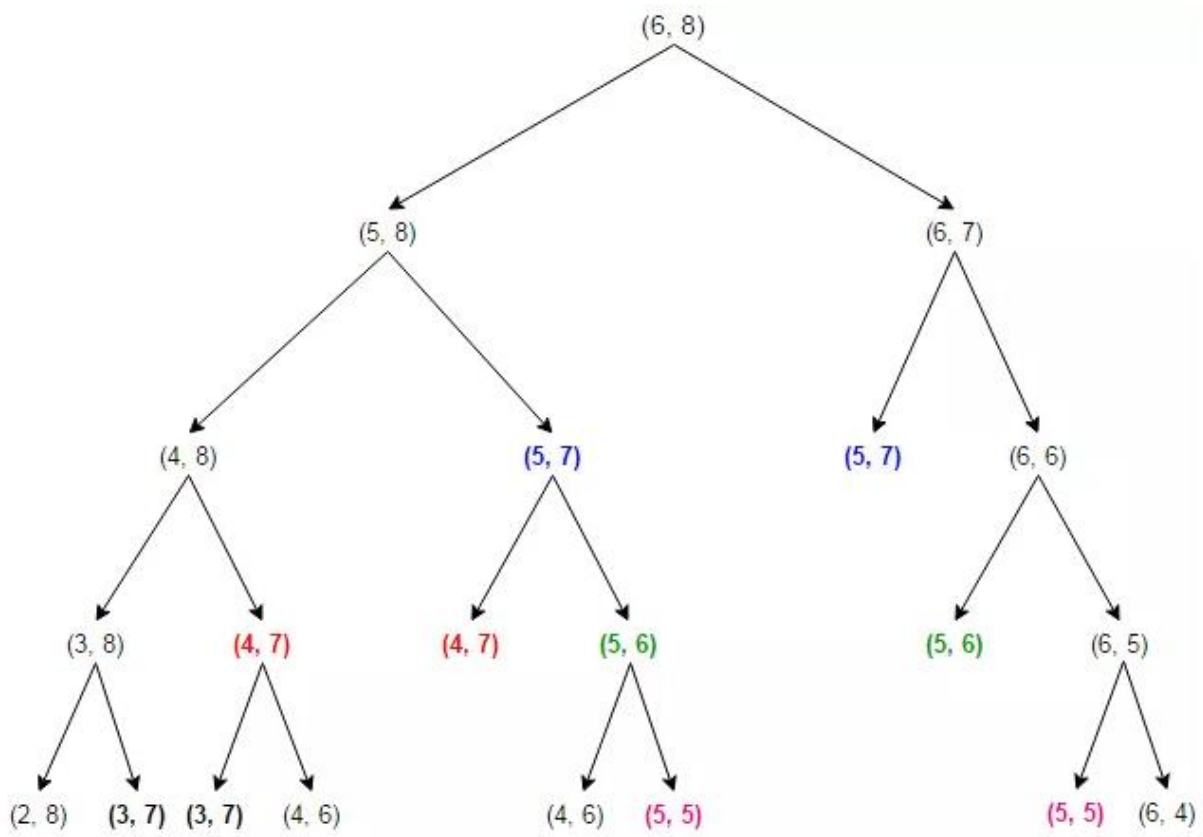
`X[0..m-1]` and `Y[0..n-1]` recursively by using optimal substructure property of SCS problem.

```
class Solution {

    int SCSUtil(string X, int M, string Y, int N){
        //if the control reaches the end of either strings
        //return the length of the other string that is remaining
        if(M==0)
            return N;
        if(N==0)
            return M;

        //if the last character of X == last character of Y then
        //call SCS(X[1...M-2], Y[1...N-2]) + 1
        if(X[M-1] == Y[N-1])
            return 1 + SCSUtil(X, M-1, Y, N-1);

        else
            return min(SCSUtil(X, M-1, Y, N) + 1,
                       SCSUtil(X, M, Y, N-1) + 1);
    }

    public:
        int SCS(string X, string Y){
            int M = X.length();
            int N = Y.length();

            return SCSUtil(X, M, Y, N);
        }
};
```

The worst case time complexity of above solution is $O(2^{(m+n)})$ and auxiliary space used by the program is `O(1).` The worst case happens when there is no common character present in X and Y (i.e. SCS length is `(m+n)`) and each recursive call will end up in two recursive calls.

Overlapping Subproblems

Let us consider recursion tree for two sequences of length 6 and 8 whose SCS length is (6+8=14). As we can see, the same sub-problems (highlighted in same color) are getting computed again and again



The problems having optimal substructure & overlapping subproblems can be solved by dynamic programming, in which subproblem solutions are memoized rather than computed again and again. Below is *Memoized* version that follows the top-down approach, as we first break the problem into subproblems and then calculate and store values.

```
int SCSTopDownUtil(string X, int M, string Y, int N, unordered_map<string,int>
                                                        &lookup)
{
        if(M==0 || N==0)
            return M+N;

        string key = to_string(M) + "|" + to_string(N);

        //if the key is not seen before then
        if(lookup.find(key) == lookup.end()){

            if(X[M-1]==Y[N-1])
                lookup[key] = SCSTopDownUtil(X, M-1, Y, N-1, lookup) + 1;
            else
                lookup[key] = min(SCSTopDownUtil(X, M-1, Y, N, lookup) + 1,
                                  SCSTopDownUtil(X, M, Y, N-1, lookup) + 1);
        }

        return lookup[key];
}


int SCSTopDown(string X, string Y){
            int M = X.length();
            int N = Y.length();

            unordered_map<string,int>lookup;

            return SCSTopDownUtil(X, M, Y, N, lookup);
}
```

The time complexity of above solution is O(mn) and auxiliary space used by the program is O(mn).

We can also solve this problem in bottom-up manner. In the bottom-up approach, we calculate the smaller values of SCS(i, j) first, then build larger values from them.

```
              | j                                      (if i == 0)
              | i                                      (if j == 0)
 SCS[i][j] = | SCS[i-1][j-1] + 1                       (if X[i-1] == Y[j-1])
              | min (SCS[i-1][j] + 1, SCS[i][j-1] + 1) (if X[i-1] != Y[j-1])
```

	Y---	G	X	T	X	A	Y	B
X |	0	1	2	3	4	5	6	7
A	1	2	3	4	5	5	6	7
G	2	2	3	4	5	6	7	8
G	3	3	4	5	6	7	8	9
T	4	4	5	5	6	7	8	9
A	5	5	6	6	7	7	8	9
B	6	6	7	7	8	8	9	9

strings X = “AGGTAB” and Y = “GXTXAYB”

SCS = AGXGTXAYB

```
int SCSBottomUp(string X, string Y)
{
      int M = X.length();
      int N = Y.length();

      int **lookup = new int*[M+1];
      for(int i=0; i<=M; i++)
            lookup[i] = new int[N+1]();

      return SCSBottomUpUtil(X, M, Y, N, lookup);
}
```

```
int SCSBottomUpUtil(string X, int M, string Y, int N, int **lookup){

        //initialize the lookup table
        for(int i=0; i<=M; i++)
            lookup[i][0] = i;

        for(int j=0; j<=N; j++)
            lookup[0][j] = j;


        //traverse the lookup and fill it up
        for(int i=1; i<=M; i++){
            for(int j=1; j<=N; j++){
                if(X[i-1] == Y[j-1])
                    lookup[i][j] = lookup[i-1][j-1] + 1;
                else
                    lookup[i][j] = min(lookup[i-1][j]+1,
                                       lookup[i][j-1]+1);
            }
        }
        return lookup[M][N];
    }
```

Printing SCS

```
      int i=M, j=N;
      string str;
      while(i>0 && j>0){
          if(X[i-1]==Y[j-1])
              str.push_back(X[i-1]), i--, j--;
          else if(DP[i-1][j] < DP[i][j-1])
              str.push_back(X[i-1]), i--;
          else
              str.push_back(Y[j-1]), j--;
      }

      //if X is not empty then add the remaining chars of X to ans
      while(i>0){
          str.push_back(X[i-1]); i--;
      }

      //if Y is not empty then add the remaining chars of Y to ans
      while(j>0){
          str.push_back(Y[j-1]); j--;
      }
      reverse(str.begin(),str.end());
```